

Experiment 4 : Impact of Principal Component Analysis on K-Nearest Neighbors Classifier Performance

Total Marks: 100

1. Learning Objectives

Upon successful completion of this assignment, students will be able to:

- Load, inspect, and clean a "messy" real-world dataset with mixed data types and missing values.
- Develop a modular data preprocessing pipeline using `scikit-learn`'s `Pipeline` and `ColumnTransformer` for imputation, encoding, and scaling.
- Analyze the impact of high-dimensionality on a K-Nearest Neighbors (KNN) classifier.
- Implement the **Principal Component Analysis (PCA)** algorithm **from scratch** using **NumPy**, including covariance matrix calculation, eigenvector decomposition, and data projection.
- Compare the "from scratch" PCA implementation against `sklearn.decomposition.PCA`.
- Evaluate and analyze the computational and performance trade-offs of using PCA as a dimensionality reduction technique before a distance-based classifier.

2. Introduction

Real-world data is messy. Before any model can be trained, data must be rigorously cleaned, transformed, and prepared. This assignment covers this critical preprocessing stage.

Furthermore, preprocessing steps like One-Hot Encoding can create datasets with hundreds of features (high-dimensionality). This "curse of dimensionality" can severely degrade the performance and speed of certain algorithms, especially distance-based ones like **K-Nearest Neighbors (KNN)**.

You will build a full preprocessing pipeline to clean the data. You will then establish a baseline by training a KNN model on the high-dimensional processed data. Next, you will implement PCA from scratch to reduce the data's dimensionality. Finally, you will compare

the performance (accuracy and speed) of three models: the baseline KNN, a KNN trained on your scratch PCA data, and a KNN trained on `sklearn`'s PCA data.

Dataset: Adult Census Income (ID: 2). The task is to predict if income > \$50K/year.

Link to Dataset: <https://archive.ics.uci.edu/dataset/2/adult>

3. Prerequisites

Ensure your Python environment has the following libraries installed:

```
pip install numpy pandas matplotlib seaborn scikit-learn ucimlrepo
```

4. Experiment Tasks

You will build a modular machine learning pipeline. Follow the tasks below.

Task 1: Project Scaffolding and Data Handling (10 Marks)

1. **Create the Project Directory:** Create a new folder for this project.
 2. **Create Python Files:** Inside the folder, create **five** empty Python files:
 - `data.py` : For loading and initial target variable preparation.
 - `preprocessing.py` : For creating the feature preprocessing pipeline.
 - `eda.py` : For scripts related to exploratory data analysis.
 - `pca_scratch.py` : For your "from scratch" PCA implementation.
 - `main.py` : The main script to execute the full pipeline and all model comparisons.
 3. **Implement the Data Loader (`data.py`):**
- Import `ucimlrepo` , `pandas` , and `sklearn.preprocessing.LabelEncoder` .
 - Use `fetch_ucirepo` to load the **Adult** dataset (ID: 2).
 - Extract features `X` (DataFrame) and targets `y` (Series).
 - **Crucial:** The dataset uses `' ? '` for missing values. Ensure these are replaced with `np.nan` .
 - The target `y` is categorical (`<=50K` , `>50K`). Use `LabelEncoder` to transform it into binary (0 and 1).

- Create a function `load_data()` that returns `X` and the encoded `y`.
-

Task 2: Exploratory Data Analysis (EDA) (15 Marks)

In `eda.py`, write a script to visualize the data's properties.

1. Implement the Visualization Script (`eda.py`):

- Import `matplotlib.pyplot`, `seaborn`, and your `load_data` function.
 - **Missing Data Analysis:** Generate a bar plot showing the percentage of missing values for every column.
 - **Numerical Analysis:** Generate histograms for continuous features (e.g., `age`, `hours-per-week`), using the target variable as the `hue`.
 - **Categorical Analysis:** Generate count plots for key categorical features (e.g., `workclass`, `occupation`), using the target variable as the `hue`.
 - Use `plt.show()` or save the plots to files.
-

Task 3: Pre-processing Pipeline Implementation (20 Marks)

In `preprocessing.py`, build a `ColumnTransformer` to handle the varied feature types.

1. Implement `get_preprocessor` (`preprocessing.py`):

- Import `SimpleImputer`, `StandardScaler`, `OneHotEncoder`, `Pipeline`, and `ColumnTransformer`.
 - Define two lists of column names (you'll need to get these from your `data.py`):
`numerical_features` and `categorical_features`.
 - **Numeric Transformer:** Create a `Pipeline` that:
 1. Imputes missing values using the **median**.
 2. Scales features using `StandardScaler` (This is **vital** for both KNN and PCA).
 - **Categorical Transformer:** Create a `Pipeline` that:
 1. Imputes missing values using the **most frequent** value.
 2. Encodes features using `OneHotEncoder` (`handle_unknown='ignore'`).
 - **Combine:** Create a `ColumnTransformer` that applies the correct transformer to each list of features.
 - Define a function `get_preprocessor()` that returns this `ColumnTransformer`.
-

Task 4: Baseline Model Evaluation (KNN) (15 Marks)

In `main.py`, you will establish your baseline by training a KNN on the *full, pre-processed, high-dimensional* data.

1. Assemble Baseline Pipeline (`main.py`):

- Import `load_data`, `get_preprocessor`, `time`, and `numpy`.
- Import `train_test_split`, `KNeighborsClassifier`, `Pipeline`, `accuracy_score`, and `classification_report`.
- Load `X` and `y`. Split into `X_train`, `X_test`, `y_train`, `y_test` (`test_size=0.2`, `random_state=42`).
- Get the pre-processor object from `get_preprocessor()`.

2. Create and Train Model 1 (Baseline):

- Create a `Pipeline` named `baseline_pipeline`. Its `steps` are:
 1. `('preprocessor', preprocessor_object)`
 2. `('classifier', KNeighborsClassifier(n_neighbors=5))`
- **Time it:** Record the start time, `baseline_pipeline.fit(X_train, y_train)`, and the end time. Print the training time.
- Record the start time, `y_pred_base = baseline_pipeline.predict(X_test)`, and the end time. Print the prediction time.
- **Evaluate:** Print the "Baseline KNN Model Accuracy" and the `classification_report`.
- **Analysis:** Get the processed `X_train` (`baseline_pipeline.named_steps['preprocessor'].transform(X_train)`) and print its `.shape`. Note this high number of features.

Task 5: PCA from Scratch (NumPy) (30 Marks)

This is the core task. In `pca_scratch.py`, you will implement PCA.

1. Implement `MyPCA` Class (`pca_scratch.py`):

- Import `numpy as np`.
- Create a class `MyPCA`.
- `__init__(self, n_components)`: Store `n_components`.
- `fit(self, X)`:

- **Note:** This method must assume `X` is *already centered and scaled* (which your Task 3 preprocessor does).
 - Compute the covariance matrix: `cov_matrix = np.cov(X.T)` . (The `.T` is crucial as `np.cov` expects features as rows).
 - Calculate eigenvalues and eigenvectors: `eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)` .
 - Sort eigenvectors: Get the indices that sort eigenvalues in descending order (`np.argsort`).
 - Store the top `n_components` eigenvectors: `self.components_ = sorted_eigenvectors[:, :self.n_components]` .
 - `transform(self, X)` :
 - Project the data: `return np.dot(X, self.components_)` .
-

Task 6: Model Integration & Comparison (10 Marks)

Back in `main.py`, you will now use your `MyPCA` and `sklearn.PCA` to train and compare two new models.

1. Get Processed Data:

- Fit your preprocessor on `X_train` and transform *both* `X_train` and `X_test` . Store these as `X_train_proc` and `X_test_proc` .

2. Find `k` from `sklearn.PCA` :

- Import `PCA` from `sklearn.decomposition` .
- Create `pca_sklearn = PCA(n_components=0.95)` (find components for 95% variance).
- Fit `pca_sklearn` on `X_train_proc` .
- Get the number of components: `k = pca_sklearn.n_components_` .
- Print a message: `sklearn PCA selected {k} components to explain 95% variance.`

3. Create and Train Model 2 (Scratch PCA):

- Import `MyPCA` from `pca_scratch.py` .
- Instantiate `my_pca = MyPCA(n_components=k)` .
- `my_pca.fit(X_train_proc)` .
- `X_train_pca_scratch = my_pca.transform(X_train_proc)` .

- `X_test_pca_scratch = my_pca.transform(X_test_proc)` .
- **Time it:** Train a new `KNeighborsClassifier(n_neighbors=5)` on this scratch-PCA data. Print the training and prediction times.
- **Evaluate:** Print the "Scratch PCA KNN Model Accuracy" and `classification_report` .

4. Create and Train Model 3 (`sklearn` PCA):

- Use the `pca_sklearn` object you already fit.
 - `X_train_pca_sklearn = pca_sklearn.transform(X_train_proc)` .
 - `X_test_pca_sklearn = pca_sklearn.transform(X_test_proc)` .
 - **Time it:** Train a *third* `KNeighborsClassifier(n_neighbors=5)` on this `sklearn` PCA data. Print the training and prediction times.
 - **Evaluate:** Print the "`sklearn` PCA KNN Model Accuracy" and `classification_report` .
-

5. Submission Guidelines

Submit a single **.zip** archive containing:

1. **Source Code:** All five completed Python files (`main.py` , `data.py` , `eda.py` , `preprocessing.py` , `pca_scratch.py`).
2. **PDF Report:** A formal report named `StudentID_Lab_Report.pdf` that includes:
 - **EDA Analysis:** A summary of your findings from Task 2 (missing data, feature distributions).
 - **Model 1 (Baseline):**
 - The total number of features after preprocessing (from Task 4).
 - The baseline model's accuracy, training time, and prediction time.
 - **PCA Analysis:**
 - The number of components (`k`) selected by `sklearn` for 95% variance.
 - **Model 2 (Scratch PCA):**
 - The scratch-PCA model's accuracy, training time, and prediction time.
 - **Model 3 (Sklearn PCA):**
 - The `sklearn` PCA model's accuracy, training time, and prediction time.

- **Comparative Analysis:**
 - Create a table comparing the **Accuracy**, **Training Time**, and **Prediction Time** for all three models.
 - Discuss the results. Why were the PCA models so much **faster** to train and predict?
 - How did the accuracy of your `MyPCA` model compare to the `sklearn.PCA` model? Were they identical? Why or why not?
- **Conclusion:** Summarize your key learning and any challenges.